_>11 Projeto Final — API de Controle Financeiro

Disciplina: Programação Web Back-end **Tipo:** Projeto avaliativo (enunciado) **Tecnologias obrigatórias:** Node.js, NPM, Express, MySQL (driver `mysql2`), autenticação com JWT (`jsonwebtoken` + `express-jwt`) e `bcrypt` **Modalidade:** grupos de até 3 pessoas **Pré-requisitos:** todos os capítulos anteriores deste livro, em especial Web Services, NPM, JSON e organização de projetos Node JS, Express Framework, Integração de APIs Node.js com Banco de Dados, APIs Node JS com JWT e Loja de carros com clientes e vendas

---

11.1 Visão geral

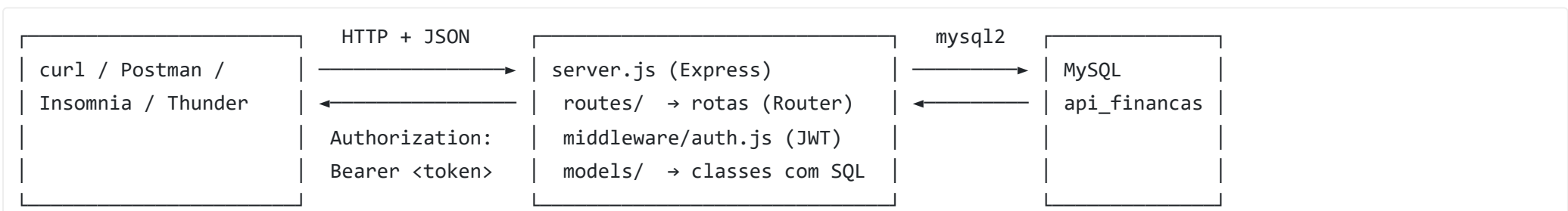
Ao longo deste livro, construímos passo a passo a **API da loja de carros**: começamos com um servidor HTTP, organizamos o projeto com o NPM, migramos para o Express, conectamos ao MySQL, protegemos as rotas com JWT e, por fim, relacionamos carros, clientes e vendas com transações.

Neste projeto, o grupo vai aplicar **os mesmos conceitos** para construir, do zero, uma **API REST de controle financeiro pessoal**. Não há interface gráfica: o produto entregue é a **API**, testada com `curl` (ou outro cliente HTTP, como Postman, Insomnia ou a extensão Thunder Client do VS Code).

Na API, um usuário se cadastra, faz *login* e registra suas movimentações de dinheiro. Cada movimentação (chamada aqui de **lançamento**) é:

- de um de **dois tipos**: **entrada** (dinheiro que chega) ou **saída** (dinheiro que sai);
- relacionada a uma **categoria**, por exemplo *alimentação*, *saúde*, *mobilidade*, *moradia* ou *salário*;
- relacionada a uma **fonte** (para entradas) ou a um **destino** (para saídas), por exemplo *salário*, *cartão Nubank*, *Visa crédito*, *poupança* ou *conta corrente*.

A partir desses dados, a API deve gerar **relatórios em JSON**: gastos por categoria, movimentação por fonte/destino e **previsão das faturas dos cartões de crédito**. Como **desafio** (pontuação extra), o grupo pode criar uma rota de **saúde financeira**, que devolve indicadores, projeções e dicas calculados a partir dos dados do usuário.



> **Comparando com a loja de carros.** Na loja, as rotas `GET` eram **públicas** e só as operações de escrita exigiam *token*. Aqui é diferente: os dados financeiros são **privados**, e **todas** as rotas (exceto cadastro e *login*) exigem *token*. Além disso, cada usuário só pode enxergar os **próprios** dados.

11.2 Objetivos de aprendizagem

Ao concluir o projeto, o(a) aluno(a) deve ser capaz de:

1. Organizar um projeto Node.js com **NPM**, *scripts* (`npm start`, `npm run dev`) e **nodemon**.
2. Construir uma **API REST** com **Express**, separando as rotas em arquivos com `express.Router()` e usando corretamente **verbos HTTP**, ***path parameters***, ***query parameters*** e **códigos de *status***.
3. **Modelar** um banco **MySQL** com chaves estrangeiras e restrições `UNIQUE`, e acessá-lo com o driver `mysql2` usando **consultas parametrizadas** (`?`).
4. Organizar o acesso a dados em **classes de modelo** (`models/`), como na classe `Carro`.
5. Implementar **cadastro** com senha protegida por ***hash*** (`bcrypt`) e ***login*** com **JWT**, protegendo as rotas com um *middleware*.
6. Usar **transações** (`beginTransaction`, `commit`, `rollback`) quando várias instruções SQL precisam ser gravadas juntas.
7. Implementar **regras de negócio** no servidor: validações, agregações (`SUM`, `GROUP BY`) e o cálculo de faturas de cartão de crédito.

11.3 Conceitos do livro aplicados no projeto

O projeto foi pensado para que **cada capítulo** do livro seja usado. A tabela abaixo serve de roteiro de estudo e de *checklist*.

Capítulo do livro	Conceito	Onde aparece no projeto
JavaScript básico e objetos	Funções, *arrow functions*, objetos, arrays (`map` , `filter` , `reduce`), *template strings*, classes	Validações, cálculo de faturas, montagem das respostas JSON, classes de modelo
Web Services	REST, recursos no plural (`/lancamentos`), verbos HTTP, códigos de *status*, `Content-Type: application/json`	Todas as rotas
Servidor HTTP com Node.js	Ciclo requisição/resposta	Base do entendimento do Express
NPM, JSON e organização	`npm init` , `package.json` , `dependencies` × `devDependencies` , `npm start` , `nodemon` , `module.exports` / `require`	Estrutura do projeto e *scripts*
Express Framework	`express.json()` , `express.Router()` , `app.use` , `req.params` , `req.query` , `req.body` , `res.status().json()`	`server.js` e pasta `routes/`
Integração com banco de dados	MySQL, `mysql2` , `database/connection.js` , SELECT/INSERT/UPDATE/DELETE com `?` , classes de modelo	Pasta `models/`
APIs com JWT	`jwt.sign` com `expiresIn` , `express-jwt` (`protegerRota`), `Authorization: Bearer` , `UnauthorizedError` , `req.auth`	Pasta `middleware/` e rota `auth.js`
JWT — Melhorando a segurança	Senha com *hash*, chave secreta fora do código, tempo de expiração	**Obrigatórios** neste projeto (no livro eram recomendações)
Loja de carros com clientes e vendas	Relacionamentos 1–N, `UNIQUE` no banco, transações, id do usuário vindo do *token*	Modelo de dados, cadastro e lançamentos

Table 11.1: Onde cada conceito do livro aparece no projeto

11.3.1 Conceitos novos (pequenos passos além do livro)

Três itens que o livro apenas **recomendou** passam a ser **obrigatórios**. Eles serão revisados em aula:

1. Variáveis de ambiente com `dotenv` . A chave secreta do JWT e a senha do banco não podem ficar no código nem no repositório.

```
npm install dotenv
```

```
# .env  (NÃO versionar — incluir no .gitignore)
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=sua_senha
DB_NAME=api_financas
JWT_SECRET=uma_frase_longa_e_dificil_de_adivinhar
```

```
// primeira linha do server.js
require('dotenv').config();

// em qualquer arquivo
const segredo = process.env.JWT_SECRET;
```

O repositório deve conter um arquivo **`.env.example`** , com os mesmos nomes de variáveis e valores fictícios, e um **`.gitignore`** com `node_modules/` e `.env` .

2. *Hash* de senha com `bcrypt` . A senha **nunca** é gravada em texto puro, e o *login* deixa de comparar a senha no SQL:

```
const bcrypt = require('bcrypt');

// no cadastro
const senhaHash = await bcrypt.hash(senha, 10);   // 10 = custo (rodadas de salt)

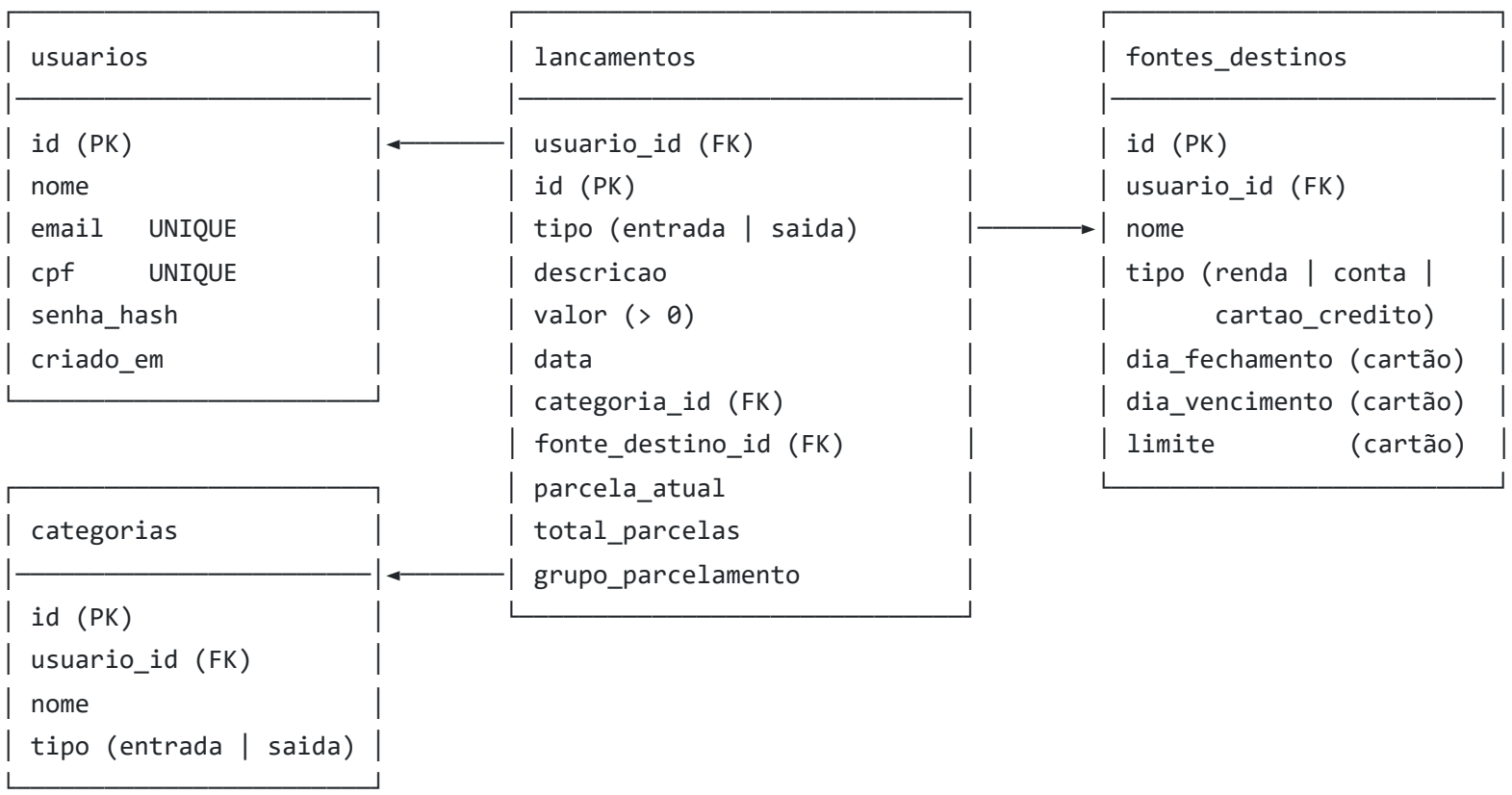
// no login: busca o usuário SÓ pelo e-mail e compara a senha no Node
const confere = await bcrypt.compare(senhaDigitada, usuario.senha_hash);
```

3. Restrições `CHECK` no MySQL. A partir do MySQL 8.0.16, as restrições `CHECK` são verificadas pelo banco. Elas impedem, por exemplo, um valor negativo, mesmo que a validação da rota falhe.

11.4 Modelo de dados

O modelo abaixo é a **referência mínima**. O grupo pode acrescentar campos e tabelas, mas **não pode remover** as entidades, os relacionamentos e as restrições descritas.

```
usuarios 1 ——— N categorias
usuarios 1 ——— N fontes_destinos
usuarios 1 ——— N lancamentos
categorias 1 —— N lancamentos
fontes_destinos 1 —— N lancamentos
```



Entidade	Descrição	Regras principais
Usuário	Pessoa que usa a API.	`email` e `cpf` **únicos no banco**; senha guardada **apenas** como *hash*.
Categoria	Classifica o lançamento (*alimentação*, *saúde*, *salário*…).	Pertence a um usuário. Tem `tipo` (`entrada` ou `saida`); um lançamento só pode usar categoria do **mesmo tipo**.
Fonte/Destino	De onde o dinheiro vem ou para onde vai (*salário*, *poupança*, *cartão Nubank*…).	Pertence a um usuário. `tipo` ∈ {`renda`, `conta`, `cartao_credito`}. Se for cartão, `dia_fechamento`, `dia_vencimento` e `limite` são **obrigatórios**.
Lançamento	Uma movimentação financeira.	`valor` **positivo** (o sinal vem do `tipo`); `data` obrigatória; **deve** ter categoria e fonte/destino do **mesmo usuário**. Compras no cartão podem ser **parceladas**.

Table 11.2: Entidades do sistema

11.4.1 Script de criação do banco

Arquivo: `database/schema.sql`

```sql
CREATE DATABASE IF NOT EXISTS api_financas;
USE api_financas;

CREATE TABLE usuarios (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nome        VARCHAR(120) NOT NULL,
  email       VARCHAR(160) NOT NULL,
  cpf         CHAR(11)     NOT NULL,                -- só dígitos
  senha_hash  VARCHAR(100) NOT NULL,
  criado_em   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_usuarios_email UNIQUE (email),
  CONSTRAINT uq_usuarios_cpf   UNIQUE (cpf)
);

CREATE TABLE categorias (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  usuario_id  INT NOT NULL,
  nome        VARCHAR(60) NOT NULL,
  tipo        ENUM('entrada', 'saida') NOT NULL,
  CONSTRAINT uq_categorias_nome UNIQUE (usuario_id, nome, tipo),
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE
);

CREATE TABLE fontes_destinos (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  usuario_id      INT NOT NULL,
  nome            VARCHAR(60) NOT NULL,
  tipo            ENUM('renda', 'conta', 'cartao_credito') NOT NULL,
  dia_fechamento  TINYINT NULL,
  dia_vencimento  TINYINT NULL,
  limite          DECIMAL(12, 2) NULL,
  CONSTRAINT uq_fontes_nome UNIQUE (usuario_id, nome),
  CONSTRAINT ck_dias CHECK (
    (dia_fechamento IS NULL OR dia_fechamento BETWEEN 1 AND 31) AND
    (dia_vencimento IS NULL OR dia_vencimento BETWEEN 1 AND 31)
  ),
  CONSTRAINT ck_cartao CHECK (
    tipo <> 'cartao_credito' OR
    (dia_fechamento IS NOT NULL AND dia_vencimento IS NOT NULL AND limite IS NOT NULL)
  ),
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE
);

CREATE TABLE lancamentos (
  id                  INT AUTO_INCREMENT PRIMARY KEY,
  usuario_id          INT NOT NULL,
  tipo                ENUM('entrada', 'saida') NOT NULL,
  descricao           VARCHAR(120)   NOT NULL,
  valor               DECIMAL(12, 2) NOT NULL,
  data                DATE           NOT NULL,
  categoria_id        INT NOT NULL,
  fonte_destino_id    INT NOT NULL,
  parcela_atual       TINYINT NOT NULL DEFAULT 1,
  total_parcelas      TINYINT NOT NULL DEFAULT 1,
  grupo_parcelamento  CHAR(36) NULL,                -- mesmo valor em todas as parcelas
  criado_em           DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT ck_valor    CHECK (valor > 0),
  CONSTRAINT ck_parcelas CHECK (total_parcelas BETWEEN 1 AND 48
                            AND parcela_atual BETWEEN 1 AND total_parcelas),
  FOREIGN KEY (usuario_id)       REFERENCES usuarios(id) ON DELETE CASCADE,
  FOREIGN KEY (categoria_id)     REFERENCES categorias(id),
  FOREIGN KEY (fonte_destino_id) REFERENCES fontes_destinos(id),
  INDEX idx_lancamentos_usuario_data (usuario_id, data)
);
```

Além do `schema.sql`, o grupo deve entregar um `database/seed.sql` com pelo menos **2 usuários** e **3 meses de lançamentos** (incluindo ao menos um cartão de crédito e uma compra parcelada), para a demonstração. Como as senhas são gravadas com *hash*, o *seed* deve conter o *hash* gerado pelo `bcrypt` (ou o grupo pode criar os usuários pela rota de cadastro e inserir o restante pelo SQL).

11.4.2 E-mail e CPF únicos: a garantia vem do banco

O sistema **não pode permitir dois usuários com o mesmo e-mail ou o mesmo CPF**. Assim como, na loja de carros, a restrição `UNIQUE` em `carro_id` garantia que um carro fosse vendido uma única vez, aqui as restrições `uq_usuarios_email` e `uq_usuarios_cpf` garantem a regra **no próprio banco**.

> **Por que não basta fazer um SELECT antes de inserir?** Se duas requisições de cadastro chegarem ao mesmo tempo, as duas podem executar o `SELECT`, não encontrar ninguém e, em seguida, as duas fazerem o `INSERT`. Esse problema se chama **condição de corrida** (*race condition*). A restrição `UNIQUE` é verificada pelo MySQL de forma atômica: a segunda inserção sempre falha. O `SELECT` prévio pode existir para dar uma resposta mais rápida, mas **a garantia é do banco**.

Para que a regra funcione de verdade, os dados precisam ser **normalizados** antes de gravar:

- e-mail: `email.trim().toLowerCase()` (assim, `Ana@Email.com` e `ana@email.com` são o mesmo e-mail);
- CPF: somente dígitos, `cpf.replace(/\D/g, '')` (assim, `529.982.247-25` e `52998224725` são o mesmo CPF).

Quando o MySQL recusa uma inserção por causa de um `UNIQUE`, o erro devolvido pelo `mysql2` tem `code === 'ER_DUP_ENTRY'` (número `1062`), e a mensagem (`sqlMessage`) contém o **nome da restrição** violada. É por isso que as restrições foram nomeadas no `schema.sql`.

11.5 Requisitos funcionais

Com exceção de RF1 e RF2, **todas** as rotas devem estar protegidas pelo *middleware* JWT. Todas as validações devem ser feitas **no servidor**, com respostas em JSON e o código de *status* adequado.

11.5.1 RF1 — Cadastro de usuário

`POST /auth/cadastro`, com corpo JSON contendo `nome`, `email`, `cpf` e `senha`.

- **Validações** (resposta `400` com as mensagens):
  - todos os campos obrigatórios;
  - e-mail em formato válido;
  - CPF com **dígitos verificadores válidos** (não basta ter 11 dígitos);
  - senha com no mínimo 8 caracteres, contendo letras e números.
- Senha gravada com `bcrypt`. O `senha_hash` **nunca** aparece em nenhuma resposta da API.
- E-mail ou CPF já cadastrado → **`409 Conflict`**, indicando **qual** campo está duplicado.
- O cadastro deve criar, **na mesma transação**, o usuário, um conjunto de **categorias padrão** (ex.: *Salário* e *Outras entradas* como entrada; *Alimentação*, *Moradia*, *Saúde*, *Mobilidade*, *Lazer*, *Educação* e *Outras saídas* como saída) e uma fonte/destino padrão (*Conta corrente*). Se qualquer inserção falhar, nada é gravado (`rollback`).
- Sucesso → **`201 Created`** com `id`, `nome` e `email`.

Função utilitária de validação de CPF:

Arquivo: `utils/cpf.js`

```
// Valida os dois dígitos verificadores do CPF
function cpfValido(cpf) {
  const d = String(cpf).replace(/\D/g, '');            // mantém só os dígitos
  if (d.length !== 11 || /^(\d)\1{10}$/.test(d)) {     // rejeita 111.111.111-11 etc.
    return false;
  }
  const digito = (tamanho) => {
    let soma = 0;
    for (let i = 0; i < tamanho; i++) soma += Number(d[i]) * (tamanho + 1 - i);
    const resto = (soma * 10) % 11;
    return resto === 10 ? 0 : resto;
  };
  return digito(9) === Number(d[9]) && digito(10) === Number(d[10]);
}

module.exports = { cpfValido };

// cpfValido('529.982.247-25') → true
// cpfValido('529.982.247-24') → false (dígito verificador errado)
```

Trecho da classe `Usuario`, com a **transação** do cadastro (mesmo padrão da classe `Venda`):

Arquivo: `models/Usuario.js` (trecho)

```
const connection = require('../database/connection');

const CATEGORIAS_PADRAO = [
  ['Salário', 'entrada'], ['Outras entradas', 'entrada'],
  ['Alimentação', 'saida'], ['Moradia', 'saida'], ['Saúde', 'saida'],
  ['Mobilidade', 'saida'], ['Lazer', 'saida'], ['Educação', 'saida'], ['Outras saídas', 'saida'],
];

class Usuario {
  cadastrar({ nome, email, cpf, senhaHash }, callback) {
    connection.beginTransaction((erro) => {
      if (erro) return callback(erro);

      const sqlUsuario = 'INSERT INTO usuarios (nome, email, cpf, senha_hash) VALUES (?, ?, ?, ?)';
      connection.query(sqlUsuario, [nome, email, cpf, senhaHash], (erro, resultado) => {
        if (erro) return connection.rollback(() => callback(erro));

        const usuarioId = resultado.insertId;
        const linhas = CATEGORIAS_PADRAO.map(([nomeCat, tipo]) => [usuarioId, nomeCat, tipo]);

        // INSERT de várias linhas: VALUES ? recebe um array de arrays
        connection.query('INSERT INTO categorias (usuario_id, nome, tipo) VALUES ?', [linhas], (erro) => {
          if (erro) return connection.rollback(() => callback(erro));

          const sqlFonte = "INSERT INTO fontes_destinos (usuario_id, nome, tipo) VALUES (?, 'Conta corrente', 'conta')";
          connection.query(sqlFonte, [usuarioId], (erro) => {
            if (erro) return connection.rollback(() => callback(erro));

            connection.commit((erro) => {
              if (erro) return connection.rollback(() => callback(erro));
              callback(null, { id: usuarioId, nome, email });
            });
          });
        });
      });
    });
  }
}

module.exports = new Usuario();
```

Trecho da rota de cadastro, tratando o `ER_DUP_ENTRY`:

Arquivo: `routes/auth.js` (trecho)

```
const express = require('express');
const bcrypt = require('bcrypt');
const Usuario = require('../models/Usuario');
const { cpfValido } = require('../utils/cpf');

const router = express.Router();

router.post('/cadastro', async (req, res) => {
  const { nome, email, cpf, senha } = req.body;
  const erros = {};

  if (!nome || !nome.trim()) erros.nome = 'Informe o nome.';
  if (!email || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) erros.email = 'E-mail inválido.';
  if (!cpf || !cpfValido(cpf)) erros.cpf = 'CPF inválido.';
  if (!senha || senha.length < 8 || !/[A-Za-z]/.test(senha) || !/\d/.test(senha)) {
    erros.senha = 'A senha deve ter ao menos 8 caracteres, com letras e números.';
  }
  if (Object.keys(erros).length > 0) {
    return res.status(400).json({ erro: 'Dados inválidos.', campos: erros });
  }

  const senhaHash = await bcrypt.hash(senha, 10);
  const dados = {
    nome: nome.trim(),
    email: email.trim().toLowerCase(),   // normalização
    cpf: cpf.replace(/\D/g, ''),         // normalização
    senhaHash,
  };

  Usuario.cadastrar(dados, (erro, usuario) => {
    if (erro && erro.code === 'ER_DUP_ENTRY') {
      const campo = erro.sqlMessage.includes('uq_usuarios_cpf') ? 'cpf' : 'email';
      return res.status(409).json({
        erro: 'Usuário já cadastrado.',
        campos: { [campo]: `Já existe um usuário com este ${campo === 'cpf' ? 'CPF' : 'e-mail'}.` },
      });
    }
    if (erro) {
      return res.status(500).json({ erro: 'Erro ao cadastrar usuário.' });
    }
    return res.status(201).json(usuario);
  });
});

module.exports = router;
```

11.5.2 RF2 — *Login*

`POST /auth/login`, com `email` e `senha`.

- Buscar o usuário **somente pelo e-mail** (normalizado) e comparar a senha com `bcrypt.compare`.
- Credenciais inválidas → **`401`** com mensagem **genérica** ("E-mail ou senha inválidos."). A resposta não pode revelar se o e-mail existe.
- Sucesso → **`200`** com o *token* gerado por `jwt.sign({ id, nome, email }, process.env.JWT_SECRET, { expiresIn: '1h' })` e os dados básicos do usuário.
- Rota `GET /auth/me` (protegida): devolve os dados do usuário dono do *token* (sem `senha_hash`).

O *middleware* segue o capítulo de JWT, agora lendo a chave do `.env`:

Arquivo: `middleware/auth.js`

```
const { expressjwt } = require('express-jwt');

const protegerRota = expressjwt({
  secret: process.env.JWT_SECRET,
  algorithms: ['HS256'],
});

module.exports = protegerRota;
```

Arquivo: `server.js` (trecho)

```
require('dotenv').config();
const express = require('express');
const protegerRota = require('./middleware/auth');

const app = express();
app.use(express.json());

app.use('/auth', require('./routes/auth'));            // cadastro, login (públicas) e /me
app.use('/categorias', protegerRota, require('./routes/categorias'));
app.use('/fontes-destinos', protegerRota, require('./routes/fontesDestinos'));
app.use('/lancamentos', protegerRota, require('./routes/lancamentos'));
app.use('/relatorios', protegerRota, require('./routes/relatorios'));

// Rota inexistente
app.use((req, res) => res.status(404).json({ erro: 'Rota não encontrada.' }));

// Tratamento de erros (sempre com 4 parâmetros e por último)
app.use((erro, req, res, next) => {
  if (erro.name === 'UnauthorizedError') {
    return res.status(401).json({ erro: 'Token ausente, inválido ou expirado.' });
  }
  console.error(erro);
  return res.status(500).json({ erro: 'Erro interno do servidor.' });
});

app.listen(process.env.PORT || 3000, () => console.log('API de finanças rodando.'));
```

> Em `routes/auth.js`, a rota `/me` deve receber o *middleware* individualmente: `router.get('/me', protegerRota, (req, res) => { ... })`.

11.5.3 RF3 — Isolamento dos dados entre usuários

Requisito **transversal**, válido para todas as rotas protegidas:

- O id do usuário vem **sempre** de `req.auth.id` (o *token*), **nunca** do corpo da requisição, da URL ou da *query string*.
- **Toda** consulta deve filtrar pelo dono: `WHERE id = ? AND usuario_id = ?`.
- Ao criar ou editar um lançamento, a API deve verificar que a **categoria** e a **fonte/destino** informadas pertencem ao **mesmo usuário**.
- Pedir um registro de outro usuário (ex.: `GET /lancamentos/57`, sendo o 57 de outra pessoa) deve devolver **`404`**: para esse usuário, o registro "não existe".

```
// ❌ ERRADO — qualquer usuário autenticado lê qualquer lançamento
connection.query('SELECT * FROM lancamentos WHERE id = ?', [id], callback);

// ✅ CERTO — o dono vem do token
connection.query('SELECT * FROM lancamentos WHERE id = ? AND usuario_id = ?', [id, usuarioId], callback);
```

11.5.4 RF4 — CRUD de categorias

Método	Rota	Descrição
`GET`	`/categorias?tipo=saida`	Lista as categorias do usuário (filtro opcional por tipo)

Método	Rota	Descrição
`POST`	`/categorias`	Cria (`201`); nome repetido para o mesmo tipo → `409`
`PUT`	`/categorias/:id`	Atualiza (`200`); inexistente ou de outro usuário → `404`
`DELETE`	`/categorias/:id`	Exclui (`204`); se houver lançamentos vinculados → `409` com mensagem explicativa

11.5.5 **RF5 — CRUD de fontes/destinos**

Mesmas rotas e regras das categorias, em `/fontes-destinos` . Além disso:

- `tipo` deve ser `renda` , `conta` ou `cartao_credito` (`400` caso contrário);
- se o tipo for `cartao_credito` , `dia_fechamento` , `dia_vencimento` (1 a 31) e `limite` (> 0) são obrigatórios (`400` se faltarem).

11.5.6 **RF6 — CRUD de lançamentos**

Método	Rota	Descrição
`GET`	`/lancamentos?inicio=2026-10-01&fim=2026-10-31&tipo=saida&categoriaId=3&fonteDestinoId=2&pagina=1`	Lista com filtros (todos opcionais; o padrão é o mês corrente) e paginação (20 por página)
`GET`	`/lancamentos/:id`	Detalha um lançamento
`POST`	`/lancamentos`	Cria um lançamento (ou várias parcelas)
`PUT`	`/lancamentos/:id`	Atualiza
`DELETE`	`/lancamentos/:id?todasParcelas=true`	Exclui a parcela ou, com `todasParcelas=true` , todas as parcelas da mesma compra

Regras:

- **Validações** (`400`): `tipo` , `descricao` , `valor` , `data` , `categoriaId` e `fonteDestinoId` obrigatórios; valor numérico **maior que zero**; data válida no formato `AAAA-MM-DD` ; o **tipo da categoria** deve ser igual ao **tipo do lançamento** (uma saída não pode usar a categoria *Salário*).
- Categoria ou fonte/destino inexistente **ou de outro usuário** → `404` .
- **Parcelamento:** se o destino for um cartão de crédito, o corpo pode trazer `parcelas` (1 a 48). A API grava **um lançamento por parcela**, todos com a **mesma `data` da compra**, o mesmo `grupo_parcelamento` (gerado com `crypto.randomUUID()` , nativo do Node) e `parcela_atual` de 1 a N. O valor é dividido **em centavos**, e a diferença de arredondamento vai para a primeira parcela: R$ 100,00 em 3× → 33,34 + 33,33 + 33,33.
- A resposta da listagem deve trazer, além dos itens, um **resumo do período filtrado**: total de entradas, total de saídas e saldo.

```
// Divide um valor em parcelas sem perder centavos
function dividirEmParcelas(valorTotal, quantidade) {
  const centavos = Math.round(valorTotal * 100);
  const base = Math.floor(centavos / quantidade);
  const resto = centavos - base * quantidade;
  return Array.from({ length: quantidade }, (_, i) => (base + (i === 0 ? resto : 0)) / 100);
}

// dividirEmParcelas(100, 3) → [33.34, 33.33, 33.33]
```

Exemplo de requisição e de resposta da listagem:

```
curl -X POST http://localhost:3000/lancamentos \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer SEU_TOKEN_AQUI" \
  -d '{"tipo":"saida","descricao":"Notebook","valor":3000,"data":"2026-10-28",
       "categoriaId":7,"fonteDestinoId":3,"parcelas":10}'
```

```
{
  "periodo": { "inicio": "2026-10-01", "fim": "2026-10-31" },
  "resumo": { "entradas": 5200.00, "saidas": 3250.00, "saldo": 1950.00 },
  "pagina": 1,
  "totalPaginas": 2,
  "itens": [
    {
      "id": 41, "tipo": "saida", "descricao": "Notebook", "valor": 300.00, "data": "2026-10-28",
      "categoria": { "id": 7, "nome": "Educação" },
      "fonteDestino": { "id": 3, "nome": "Cartão Nubank" },
      "parcela": "1/10"
    }
  ]
}
```

11.5.7 RF7 — Relatório por categoria

`GET /relatorios/categorias?inicio=AAAA-MM-DD&fim=AAAA-MM-DD`

- Total de **saídas por categoria**, com o **percentual** de cada uma sobre o total, do maior para o menor.
- Total de **entradas por categoria**.
- **Comparação com o período anterior** de mesmo tamanho (ex.: outubro × setembro): para cada categoria, o valor do período anterior e a variação percentual.
- `inicio` e `fim` obrigatórios e válidos, com `inicio <= fim` (`400` caso contrário).

Os totais devem ser calculados **com SQL** (`SUM` e `GROUP BY`), e não carregando todos os lançamentos para somar no JavaScript:

```
SELECT c.id, c.nome, SUM(l.valor) AS total
  FROM lancamentos l
  JOIN categorias  c ON c.id = l.categoria_id
 WHERE l.usuario_id = ?
   AND l.tipo = 'saida'
   AND l.data BETWEEN ? AND ?
 GROUP BY c.id, c.nome
 ORDER BY total DESC;
```

> **Atenção:** o `mysql2` devolve colunas `DECIMAL` e o resultado de `SUM` como **texto** (`"980.50"`), para não perder precisão. Converta com `Number(...)` antes de calcular percentuais e de devolver o JSON.

Exemplo de resposta:

```
{
  "periodo": { "inicio": "2026-10-01", "fim": "2026-10-31" },
  "periodoAnterior": { "inicio": "2026-09-01", "fim": "2026-09-30" },
  "saidas": {
    "total": 3250.00,
    "categorias": [
      { "id": 4, "nome": "Moradia",     "total": 1500.00, "percentual": 46.15, "anterior": 1500.00, "variacao": 0.00 },
      { "id": 3, "nome": "Alimentação", "total": 980.50,  "percentual": 30.17, "anterior": 870.00,  "variacao": 12.70 },
      { "id": 6, "nome": "Mobilidade",  "total": 769.50,  "percentual": 23.68, "anterior": 812.30,  "variacao": -5.27 }
    ]
  },
  "entradas": {
    "total": 5200.00,
    "categorias": [ { "id": 1, "nome": "Salário", "total": 5200.00, "percentual": 100.00 } ]
  }
}
```

11.5.8 RF8 — Relatório por fonte/destino

`GET /relatorios/fontes-destinos?inicio=AAAA-MM-DD&fim=AAAA-MM-DD`

- Total de **entradas por fonte** (quanto veio do salário, de *freelances*, de rendimentos...).

- Total de **saídas por destino** (quanto saiu da conta corrente, de cada cartão, da poupança...).
- Para fontes/destinos do tipo `conta`: **saldo do período** (entradas − saídas).

11.5.9 RF9 — Previsão de faturas de cartão de crédito

```
GET /relatorios/faturas?meses=3
```

Para **cada cartão de crédito** do usuário, devolver a **fatura atual** (aberta) e as **próximas faturas** (o padrão é 3; `meses` aceita de 1 a 12), considerando as **parcelas futuras**. Para cada fatura:

- data de **fechamento**, data de **vencimento** e **referência** (`AAAA-MM`, mês do vencimento);
- os lançamentos que a compõem, com a parcela (ex.: `"Notebook — 3/10"`);
- o **valor total**.

Para cada cartão, também: **limite total**, **limite utilizado** (soma de todas as parcelas cuja fatura ainda não venceu) e **limite disponível**.

Regra de fechamento. Uma compra entra na fatura que fecha **no mesmo mês** se for feita **antes** do dia de fechamento; se for feita **no dia do fechamento ou depois**, entra na fatura do mês seguinte. A parcela k cai $k - 1$ meses depois da fatura da primeira parcela. Se o dia de vencimento for **maior** que o de fechamento, o vencimento é no mesmo mês do fechamento; senão, no mês seguinte. Dias que não existem no mês (ex.: dia 31 em fevereiro) viram o último dia do mês.

Cartão	Compra	Parcela	Fechamento	Vencimento	Referência
Nubank (fecha dia 3, vence dia 10)	02/11/2026	1/1	03/11/2026	10/11/2026	2026-11
Nubank (fecha dia 3, vence dia 10)	03/11/2026	1/1	03/12/2026	10/12/2026	2026-12
Visa (fecha dia 25, vence dia 5)	28/12/2026	1/1	25/01/2027	05/02/2027	2027-02
Visa (fecha dia 25, vence dia 5)	20/12/2026	3/3	25/02/2027	05/03/2027	2027-03

Table 11.3: Exemplos de aplicação da regra de fechamento

Implementação de referência (o grupo pode usá-la, adaptá-la ou escrever a sua):

Arquivo: `utils/faturas.js`

```
// Datas em UTC para evitar problemas de fuso horário
function ultimoDiaDoMes(ano, mes) {              // mes: 0 a 11
  return new Date(Date.UTC(ano, mes + 1, 0)).getUTCDate();
}

function dataSegura(ano, mes, dia) {             // aceita mes > 11 e ajusta dia 31 em meses curtos
  const a = ano + Math.floor(mes / 12);
  const m = ((mes % 12) + 12) % 12;
  return new Date(Date.UTC(a, m, Math.min(dia, ultimoDiaDoMes(a, m))));
}

// dataCompra: 'AAAA-MM-DD'; cartao: { diaFechamento, diaVencimento }
function calcularFatura(dataCompra, cartao, numeroParcela = 1) {
  const [ano, mes, dia] = dataCompra.split('-').map(Number);
  const m = mes - 1;
  const fechamentoDoMes = Math.min(cartao.diaFechamento, ultimoDiaDoMes(ano, m));

  let mesFechamento = dia < fechamentoDoMes ? m : m + 1;   // antes do fechamento: fatura deste mês
  mesFechamento += numeroParcela - 1;                       // parcelas seguintes: meses seguintes

  const fechamento = dataSegura(ano, mesFechamento, cartao.diaFechamento);
  const mesVencimento = cartao.diaVencimento > cartao.diaFechamento ? mesFechamento : mesFechamento + 1;
  const vencimento = dataSegura(ano, mesVencimento, cartao.diaVencimento);

  const iso = (d) => d.toISOString().slice(0, 10);
  return { referencia: iso(vencimento).slice(0, 7), fechamento: iso(fechamento), vencimento: iso(vencimento) };
}

module.exports = { calcularFatura };

// calcularFatura('2026-12-28', { diaFechamento: 25, diaVencimento: 5 })
// → { referencia: '2027-02', fechamento: '2027-01-25', vencimento: '2027-02-05' }
```

Dica de implementação. Busque no banco todos os lançamentos de saída de cada cartão, calcule a fatura de cada um com `calcularFatura(data, cartao, parcela_atual)` e agrupe os resultados por `referencia` com `reduce`. Depois, mantenha só as faturas a partir da atual. Isso exercita bem os métodos de array do capítulo de JavaScript.

Datas no `mysql2`. Por padrão, colunas `DATE` chegam como objetos `Date`, sujeitos ao fuso horário. Para receber o texto `'AAAA-MM-DD'`, use a opção `dateStrings: true` na configuração da conexão.

11.5.10 Desafio (pontuação extra) — Saúde financeira

`GET /saude-financeira`

Uma rota que **analisa os dados do próprio usuário** e devolve, em JSON, **indicadores**, **projeções** e **dicas**. Não há resposta única: o grupo escolhe as análises. Sugestões:

Indicador	Como calcular (sugestão)	Exemplo de dica
Taxa de poupança	(entradas − saídas) ÷ entradas, por mês	"Você guardou 4% da sua renda em outubro. Uma meta comum é guardar pelo menos 10%."
Comprometimento com cartões	Total das próximas faturas ÷ renda média mensal	"Suas faturas dos próximos 3 meses comprometem 62% da sua renda."
Parcelas futuras	Soma das parcelas a vencer, por mês	"Você tem R$ 1.240,00 em parcelas até março de 2027."
Regra 50/30/20	Classificar as categorias em *necessidades*, *desejos* e *poupança* e comparar com 50% / 30% / 20%	"Os gastos com desejos estão em 41% (referência: 30%)."

Indicador	Como calcular (sugestão)	Exemplo de dica
Projeção de saldo	Média das entradas e saídas dos últimos 3 meses + parcelas já conhecidas, para os próximos 3 a 6 meses	Lista mês a mês com o saldo previsto e um alerta se ficar negativo
Categoria fora da curva	Gasto do mês na categoria × média dos 3 meses anteriores	"Seus gastos com Alimentação estão 35% acima da sua média."
Reserva de emergência	Saldo das contas marcadas como reserva (sugere-se um campo `reserva` em `fontes_destinos`) ÷ média de saídas mensais	"Sua reserva cobre 1,8 mês de despesas. Uma referência comum é de 3 a 6 meses."

Table 11.4: Sugestões de indicadores

```
{
  "aviso": "Informações educativas; não constituem recomendação financeira profissional.",
  "indicadores": {
    "taxaPoupanca": 0.04,
    "comprometimentoCartoes": 0.62,
    "mesesDeReserva": 1.8
  },
  "projecao": [
    { "mes": "2026-11", "entradas": 5200.00, "saidas": 4950.00, "saldoPrevisto": 250.00 },
    { "mes": "2026-12", "entradas": 5200.00, "saidas": 5410.00, "saldoPrevisto": -210.00 }
  ],
  "dicas": [
    "Seus gastos com Alimentação estão 35% acima da sua média dos últimos 3 meses.",
    "Seu saldo previsto para dezembro é negativo: revise as compras parceladas."
  ]
}
```

As dicas podem ser geradas por **regras** (`if` com limites definidos pelo grupo e documentados no README). A resposta deve conter o campo `aviso` informando que as informações são **educativas** e **não constituem recomendação financeira profissional**.

11.6 Padrão de respostas e códigos de *status*

Todas as respostas de erro devem ter **o mesmo formato**:

```
{
  "erro": "Dados inválidos.",
  "campos": {
    "valor": "O valor deve ser maior que zero.",
    "categoriaId": "A categoria informada é de entrada, mas o lançamento é de saída."
  }
}
```

Código	Quando usar neste projeto
`200 OK`	Consultas, atualização e *login* bem-sucedidos
`201 Created`	Cadastro de usuário, categoria, fonte/destino e lançamento
`204 No Content`	Exclusão bem-sucedida (sem corpo)
`400 Bad Request`	Campos obrigatórios ausentes, formatos inválidos, regras de negócio violadas
`401 Unauthorized`	*Token* ausente, inválido ou expirado; *login* com credenciais erradas
`404 Not Found`	Registro inexistente **ou de outro usuário**; rota inexistente
`409 Conflict`	E-mail/CPF já cadastrado; nome repetido; exclusão de registro com lançamentos vinculados

Código	Quando usar neste projeto
`500 Internal Server Error`	Erros inesperados (o detalhe vai para o `console.error`, não para o cliente)

Table 11.5: Códigos de *status* esperados

11.7 **Estrutura do projeto**

A estrutura segue a da loja de carros, acrescida de `utils/` e dos arquivos de configuração:

```
api-financas/
├── database/
│   ├── connection.js       → conexão com o MySQL (lendo o .env)
│   ├── schema.sql          → criação do banco e das tabelas
│   └── seed.sql            → dados para teste e demonstração
├── middleware/
│   └── auth.js             → protegerRota (express-jwt)
├── models/                 → classes com o SQL (Usuario, Categoria, FonteDestino, Lancamento, Relatorio)
├── routes/                 → um Router por recurso (auth, categorias, fontesDestinos, lancamentos, relatorios)
├── utils/                  → funções puras (cpf.js, faturas.js, parcelas)
├── testes/
│   └── requisicoes.md      → comandos curl (ou coleção Postman/Insomnia exportada) para TODAS as rotas
├── .env.example
├── .gitignore              → node_modules/ e .env
├── package.json
├── README.md
└── server.js
```

Regras:

- Sistema de módulos **CommonJS** (`require` / `module.exports`), como em todo o livro.
- **SQL apenas em `models/`**. As rotas validam os dados, chamam os métodos das classes e montam a resposta.
- *Scripts* no `package.json`: `"start": "node server.js"` e `"dev": "nodemon server.js"` (com o `nodemon` em `devDependencies`).
- `database/connection.js` deve ler `DB_HOST`, `DB_USER`, `DB_PASSWORD` e `DB_NAME` do `.env`. O uso de `mysql.createPool(...)` no lugar de `createConnection` é **opcional** e recomendado. Atenção: com *pool*, a transação deve ser feita em uma conexão obtida com `pool.getConnection(...)`.
- O uso de `async` / `await` com a interface de *promises* do `mysql2` (`connection.promise()` ou `require('mysql2/promise')`) é **permitido**, no lugar dos *callbacks* usados no livro.

11.8 **Política acadêmica**

- Bibliotecas além de `express`, `mysql2`, `jsonwebtoken`, `express-jwt`, `bcrypt`, `dotenv` e `nodemon` precisam de **aprovação prévia** do professor; caso contrário, serão desconsideradas da avaliação.
- Se for constatada **cópia de artefatos** de outros projetos, será atribuída **nota zero**, sem direito à etapa de recuperação.
- O uso de assistentes de IA é permitido como apoio, mas **ambos os integrantes devem ser capazes de explicar qualquer trecho** do código durante a apresentação. A incapacidade de explicar o código entregue será tratada como ausência de autoria daquele trecho.

11.9 **Entrega e apresentação**

A entrega consiste no **link do repositório Git**, com *commits* incrementais dos dois integrantes, e em uma **apresentação de no máximo 10 minutos**, contemplando obrigatoriamente:

1. **Código-fonte:** estrutura de pastas, `schema.sql`, uma classe de modelo, um arquivo de rotas e o *middleware*.
2. **Execução pela linha de comando:** criação do banco (`schema.sql + seed.sql`), `npm install` e `npm run dev`.

3. **Cadastro e unicidade:** cadastro de um usuário e tentativas de cadastro com o **mesmo e-mail** (com letras maiúsculas) e com o **mesmo CPF** (com pontuação), ambas rejeitadas com `409`.
4. ***Login* e proteção:** *login*, requisição **sem** *token* (`401`) e tentativa de ler, alterar e excluir um lançamento **de outro usuário** (`404`).
5. **CRUD:** categorias, fontes/destinos e lançamentos, incluindo uma **compra parcelada** no cartão e uma validação que falha (ex.: saída com categoria de entrada).
6. **Relatórios:** por categoria, por fonte/destino e previsão de faturas, mostrando em qual fatura caiu cada parcela da compra do item 5.
7. **Desafio** (se implementado): rota de saúde financeira.

11.10 Critérios de avaliação

A nota do projeto é de **100 pontos**, mais até **10 pontos extras** pelo desafio.

#	Critério	Pontos
	A. Usuários e autenticação	**25**
1	Cadastro com validações no servidor (campos, e-mail, CPF com dígitos verificadores, senha) e transação com categorias/fonte padrão	6
2	E-mail e CPF únicos garantidos por `UNIQUE` no banco, com normalização e resposta `409` indicando o campo	6
3	Senha com `bcrypt`, *login* com JWT e expiração, chave secreta no `.env`	7
4	Rotas protegidas com `express-jwt`, tratamento do `UnauthorizedError` (`401`) e isolamento dos dados entre usuários (RF3)	6
	B. Cadastros e lançamentos	**25**
5	CRUD de categorias e de fontes/destinos com as regras de tipo, cartão, duplicidade e exclusão	8
6	CRUD de lançamentos com validações (valor, data, compatibilidade de tipo, posse da categoria e da fonte/destino)	9
7	Filtros por *query parameters*, paginação e resumo (entradas, saídas e saldo)	4
8	Compras parceladas, com divisão correta dos centavos e exclusão de todas as parcelas	4
	C. Relatórios	**20**
9	Relatório por categoria com `SUM` / `GROUP BY`, percentuais e comparação com o período anterior	7
10	Relatório por fonte/destino (entradas, saídas e saldo das contas)	5
11	Previsão de faturas (regra de fechamento, parcelas futuras, limite utilizado e disponível)	8
	D. Organização e qualidade	**15**
12	Estrutura de pastas, `express.Router()` por recurso, SQL somente nas classes de `models/`, CommonJS	4

#	Critério	Pontos
13	Verbos HTTP, códigos de *status* e padrão único de resposta de erro	4
14	Consultas parametrizadas (`?`), restrições no banco (FK, `UNIQUE`, `CHECK`) e tipos adequados para dinheiro	4
15	`package.json` com *scripts* e `nodemon`, `.env.example`, `.gitignore`, `schema.sql`, `seed.sql`, README e arquivo de testes com todas as rotas	3
	E. Apresentação	**15**
16	Explicação do código-fonte (ambos os integrantes)	5
17	Execução pela linha de comando (banco e API)	5
18	Demonstração de todos os itens obrigatórios do roteiro, dentro do tempo	5
	TOTAL	**100**
	Desafio — Saúde financeira (pelo menos 3 indicadores, uma projeção mês a mês e dicas geradas a partir dos dados)	**+10**

Table 11.6: Critérios de avaliação

11.11 Sugestão de cronograma

Marco	Entregável esperado
1. Projeto e banco	`npm init`, dependências, *scripts*, `.env`, `schema.sql`, `connection.js` e `server.js` respondendo.
2. Autenticação	Cadastro (com `UNIQUE` e transação), *login*, `protegerRota` e `/auth/me`.
3. CRUD	Categorias, fontes/destinos e lançamentos (com parcelamento), com validações e isolamento.
4. Relatórios	Relatórios por categoria e por fonte/destino; previsão de faturas.
5. Acabamento	`seed.sql`, arquivo de testes, README, desafio (opcional) e ensaio da apresentação.

Table 11.7: Marcos sugeridos (as datas oficiais estão no ambiente virtual da disciplina)

11.12 *Checklist* final do grupo

- ☐ Cadastrar o mesmo e-mail com letras maiúsculas diferentes é rejeitado com `409`.
- ☐ Cadastrar o mesmo CPF com e sem pontuação é rejeitado com `409`.
- ☐ Mesmo sem o `SELECT` prévio no código, o banco impede duplicatas.
- ☐ Nenhuma resposta da API contém `senha_hash`.
- ☐ Uma requisição sem *token*, ou com *token* expirado, recebe `401` em JSON (e não uma página de erro HTML).

- ☐ Trocar o id na URL não dá acesso a dados de outro usuário.
- ☐ Um lançamento de **saída** não aceita categoria de **entrada**.
- ☐ Uma compra de R$ 100,00 em 3× gera parcelas que somam exatamente R$ 100,00.
- ☐ Uma compra feita **no dia do fechamento** aparece na fatura do mês seguinte.
- ☐ Os relatórios devolvem números (`980.5`), e não textos (`"980.50"`).
- ☐ O `.env` **não** está no repositório; o `.env.example` está.
- ☐ Seguindo apenas o README, outra pessoa consegue criar o banco e subir a API.

11.13 Referências

- Express: Roteamento, Usando *middlewares*, Tratamento de erros
- mysql2 — documentação (consultas parametrizadas, *pool*, transações e opção `dateStrings`)
- MySQL: Restrições `CHECK` e Mensagens de erro do servidor (`ER_DUP_ENTRY` , 1062)
- jsonwebtoken, express-jwt, bcrypt, dotenv
- OWASP: *Password Storage Cheat Sheet*

« 10 APIs Node JS com JWT

12 Loja de carros com clientes e vendas »

"**Desenvolvimento web Avançado**" was written by Juliana Costa Silva. It was last built on 2026-11-06.

This book was built by the bookdown R package.